

BTS SIO - Option SLAM

Session 2026

Réalisation Professionnelle No. 2

RAPPORT DE PROJET

VOUCHER MANAGEMENT SYSTEM (VMS)

Application de gestion de bons cadeau multi-magasins

MCCI Business School

Cybercité, Ebène - Île Maurice

SOMMAIRE

1. Introduction	3
a) Description de l'application	
b) Fonctionnement de l'application	
2. Objectifs des Missions	4
3. Calendrier de Réalisation	5
a) Les étapes de réalisation	
b) Les rôles et responsabilités	
c) Maquettes	
d) Diagramme d'activité	
4. Conception (structure des écrans)	9
a) MCD (Modèle Conceptuel de Données)	
b) Description des écrans	
5. Développement	15
a) Les langages et technologies	
b) Architecture et organisation du code	
c) Outils utilisés	
d) Explication des fonctionnalités	
6. Conclusion	21
7. Annexe	22
a) Références utilisées	
b) Apprentissage	

1 – INTRODUCTION

a) Description de l'application

Le Voucher Management System, communément appelé VMS, est une application bureau pensée pour centraliser la gestion des bons cadeau d'un groupe commercial exploitant plusieurs magasins. L'outil couvre tout le cycle de vie du bon, depuis la demande passée par un client entreprise jusqu'à sa rédemption en caisse par le porteur du bon, en passant par le paiement, l'approbation, l'émission et le reporting.

L'objectif est de doter le siège d'un outil capable d'émettre des bons de manière sécurisée et traçable, d'envoyer automatiquement les PDF aux clients par email, et de permettre aux magasins partenaires de valider les bons présentés grâce à un système de QR code unique. L'application a été développée en JavaFX pour la partie bureau, avec une base de données PostgreSQL hébergée dans le cloud chez AlwaysData.

b) Fonctionnement de l'application

- Gestion des demandes de bons :

Un comptable ou un administrateur peut enregistrer une demande pour le compte d'un client entreprise. Chaque demande précise le client concerné, le nombre de bons souhaités et la valeur unitaire. Une référence unique de type VR0048-200 est générée automatiquement par le système.

- Validation du paiement :

Une fois la demande enregistrée, elle passe au statut en attente de paiement. Le comptable valide ensuite le paiement côté système, ce qui déclenche une notification à l'approbateur.

- Approbation de la demande :

L'approbateur vérifie la demande payée et la valide. L'approbation ouvre la voie à la génération des bons.

- Émission des vouchers :

Au moment de l'approbation, le système génère un voucher par bon demandé. Chaque voucher a son propre code, son QR code unique, sa valeur fixe et une date d'expiration. Un PDF est produit pour chaque voucher, et une option PDF Groupe permet d'obtenir un document unique regroupant tous les bons d'une même demande.

- Envoi automatique par email :

Les bons générés sont envoyés automatiquement au client par email, avec le PDF en pièce jointe. L'envoi passe par un serveur SMTP Gmail configuré avec un mot de passe d'application dédié.

- R  d  mption en magasin :

Dans un magasin partenaire, le superviseur scanne le QR code du voucher. Le syst  me interroge la base centrale pour v  rifier que le bon est bien valide, non expir  , non d  j   utilis  . Si tout est bon, la r  d  mption est enregistr  e avec la date, l'heure et le magasin concern  .

- Reporting Excel :

Depuis le tableau de bord, un bouton permet d'exporter un rapport Excel    trois onglets : toutes les demandes, les demandes en attente de paiement, et la liste compl  te des vouchers avec leur statut actuel.

2 – OBJECTIFS DES MISSIONS

→ Centraliser la gestion des bons cadeau : l'enjeu principal était de remplacer une gestion dispersée (tableurs, emails, bouts de papier) par une application unique où toutes les données des clients, des demandes, des paiements, des bons et des rédemptions sont stockées dans une base PostgreSQL commune, accessible par les différents profils d'utilisateurs.

→ Sécuriser le processus d'émission : chaque voucher doit avoir un code unique, un QR code unique, une valeur fixe et une date d'expiration claire. Un bon ne doit jamais pouvoir être utilisé deux fois, ni falsifié, ni régénéré en double.

→ Automatiser les étapes pénibles : la génération des PDF, l'envoi des emails aux clients, la mise à jour des statuts à chaque étape, tout cela doit se faire sans intervention manuelle répétitive. L'utilisateur clique sur approuver et le système fait le reste.

→ Permettre une rédemption rapide en magasin : le superviseur en caisse doit pouvoir scanner un QR code et obtenir une réponse immédiate. Pas question de devoir consulter un tableur ou téléphoner au siège pour vérifier si un bon est valide.

→ Assurer la traçabilité : chaque action importante doit laisser une trace. Qui a créé la demande, qui a validé le paiement, qui a approuvé, quand le voucher a été émis, quand il a été rédimé et dans quel magasin. Cette traçabilité est indispensable pour l'audit et pour désamorcer les litiges.

→ Fournir un vrai reporting : la direction doit pouvoir tirer à tout moment un état clair de la situation. Combien de demandes en attente de paiement, combien de bons émis, combien rédimés, combien proches de l'expiration. Ce reporting passe par un export Excel généré directement depuis l'application.

3 – CALENDRIER DE RÉALISATION

a) Les étapes de réalisation

- Analyse des besoins :

La première étape a consisté à lire attentivement le contexte fourni par le chargé de cours pour bien cerner le périmètre du projet : quels acteurs, quelles règles de gestion, quelles contraintes techniques. J'ai également cherché à comprendre le flux métier dans son ensemble, en me mettant tour à tour dans la peau du comptable, de l'approbateur et du superviseur magasin, pour identifier les informations dont chaque rôle a besoin à chaque étape.

- Conception et planification :

Une fois les besoins posés, j'ai travaillé sur la conception technique. Cela a demandé de modéliser la base de données (quelles tables, quelles relations, quelles contraintes), de prévoir l'architecture de l'application JavaFX (organisation des controllers, des DAO, des vues FXML) et de choisir les bibliothèques externes comme ZXing pour les QR codes, PDFBox pour les PDF, javax.mail pour l'envoi SMTP et Apache POI pour Excel.

- Développement :

Le développement a été découpé par module. J'ai commencé par la couche base avec la connexion PostgreSQL et les DAO, puis je suis monté progressivement vers les interfaces. L'ordre a suivi le cycle de vie du bon : gestion des clients et magasins, puis les demandes, puis le paiement et l'approbation, puis la génération des vouchers avec QR code et PDF, puis l'envoi email, puis la rédemption, et enfin le reporting Excel.

- Tests et corrections :

À chaque fonctionnalité terminée, j'ai effectué des tests fonctionnels pour m'assurer que le flux marchait de bout en bout. Certains bugs ont demandé plusieurs passages, notamment la connexion à la base qui se reconnectait à chaque action (résolu en sortant la connexion du try-with-resources) et les conflits entre les bibliothèques Jakarta Mail et javax Mail pour l'envoi SMTP.

- Mise en production :

Le projet a été empaqueté en application native Windows grâce à l'outil jpackage, avec une classe Launcher dédiée pour contourner la limite de jpackage qui refuse les classes étendant Application. La base PostgreSQL tourne dans le cloud chez AlwaysData et l'accès se fait depuis n'importe quelle machine disposant de l'exécutable.

- Maintenance et améliorations :

Une fois la base du projet fonctionnelle, plusieurs améliorations ont été apportées : refactorisation des DAO avec des méthodes helper pour extraire les résultats et des constantes SQL réutilisables, ajout de la fonctionnalité PDF Groupe pour regrouper tous les bons d'une demande, mise en place du reporting Excel via trois vues PostgreSQL dédiées, et correction du défilement sur l'écran de rédemption.

b) Les rôles et responsabilités

Le projet a été démarré en première année au sein d'une équipe de 3 à 4 personnes, avec une répartition classique entre la partie base de données, la partie interface JavaFX et la partie intégration (PDF, emails, QR codes). À cette période, les réunions régulières et un tableau Trello partagé servaient à garder une vue d'ensemble sur qui faisait quoi.

En deuxième année, j'ai repris le projet seul pour le stabiliser et le faire évoluer. Ce passage d'un travail en équipe à un travail individuel m'a permis de reprendre l'ensemble du code avec du recul, de repenser l'architecture des DAO, de refactoriser les parties qui posaient problème et d'ajouter les fonctionnalités qui manquaient comme le PDF Groupe et le reporting Excel. Toutes les tâches ont été reprises individuellement, du refactoring à la documentation en passant par les corrections de bugs SMTP et de connexion base.

Ce changement de configuration a aussi été l'occasion d'apprendre à relire du code qu'on n'a pas forcément écrit soi-même au départ, à identifier ce qui tient debout et ce qui doit être revu, et à prendre des décisions seul sans pouvoir se reposer sur l'avis d'un collègue.

c) Maquettes

Avant de partir dans le développement de l'interface JavaFX, j'ai réalisé deux maquettes basse fidélité des écrans principaux pour valider la disposition des éléments. L'idée n'était pas de figurer le design final, mais juste de poser où seraient les menus, les tableaux de données, les boutons d'action et les formulaires. Le design définitif a ensuite évolué au fur et à mesure du développement, mais ces maquettes ont servi de base de départ.

Voici les deux maquettes réalisées :

Maquette 1 - Écran de connexion :

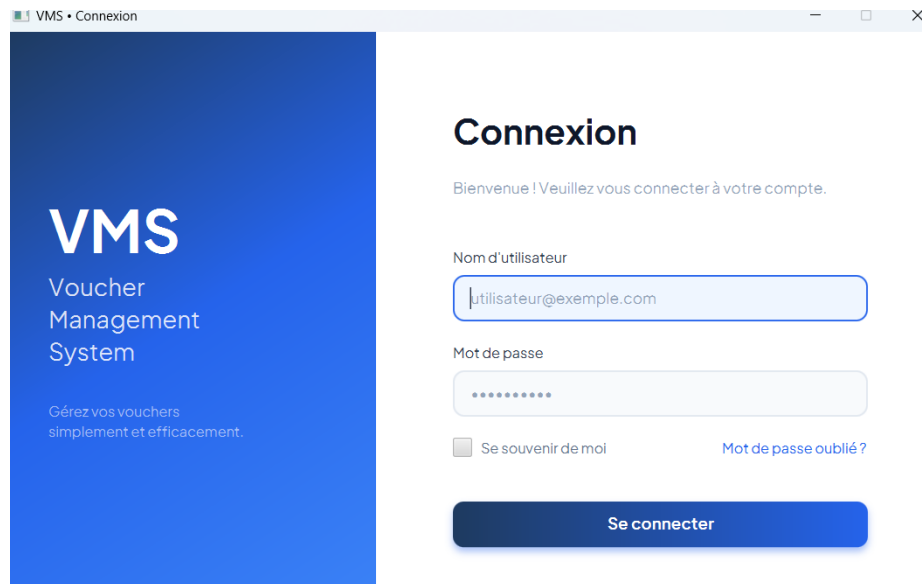


Figure 1 - Maquette de l'écran de connexion

Maquette 2 - Tableau de bord principal :

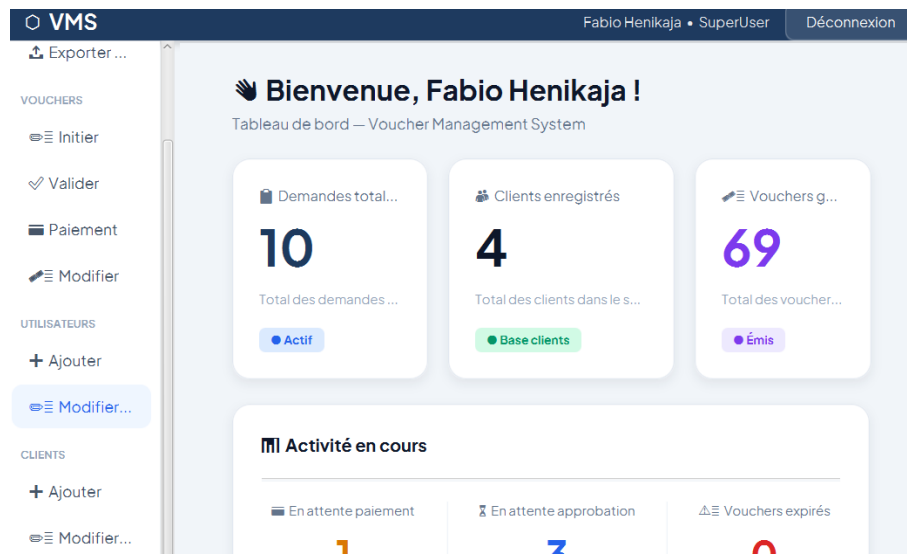


Figure 2 - Maquette du tableau de bord principal

d) Diagramme d'activité

En complément du Gantt et du PERT, un diagramme d'activité a été réalisé pour visualiser clairement le flux métier complet du bon cadeau, depuis la création de la demande jusqu'à la rédemption en magasin. Ce diagramme sert de fil conducteur pour l'implémentation des écrans et des controllers : chaque boîte correspond à une action réelle de l'application et chaque transition à un changement de statut dans la base.

On y retrouve les deux points de décision majeurs du flux : l'approbation de la demande (qui peut la valider et déclencher la génération des vouchers, ou l'annuler) et la vérification au moment du scan en magasin (voucher valide et non expiré, ou rejet). Les actions utilisateur sont différenciées des changements d'état automatiques générés par le système.

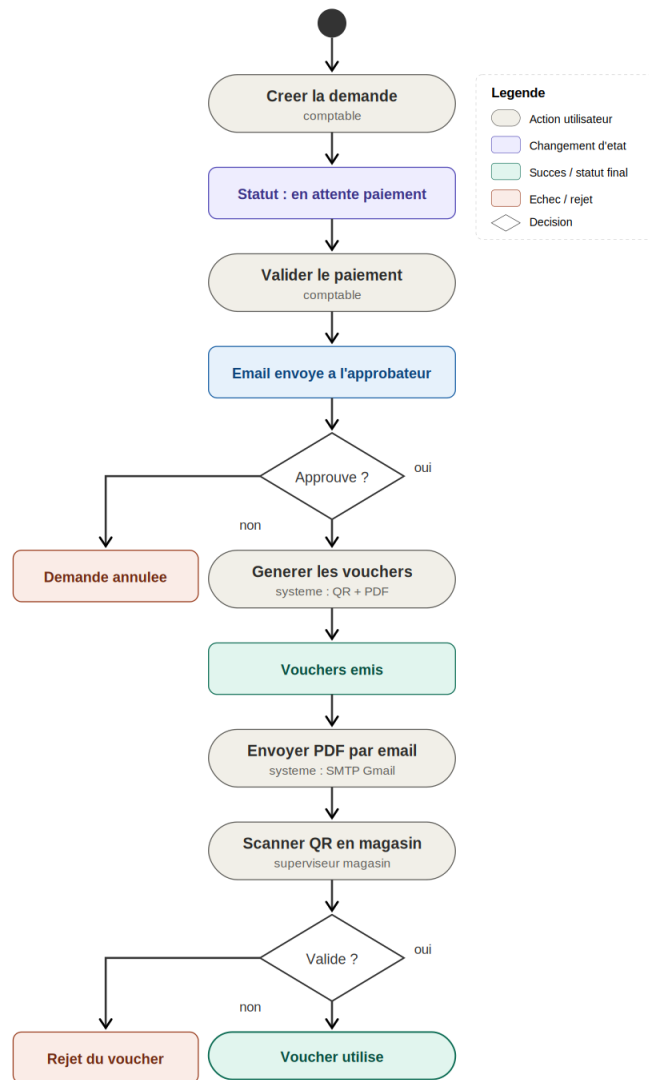


Figure 3 - Diagramme d'activité du flux voucher

4 – CONCEPTION (structure des écrans)

a) MCD (Modèle Conceptuel de Données)

Le Modèle Conceptuel de Données représente de façon abstraite les entités manipulées par le projet et les liens qui les unissent. C'est la base qui a servi à générer le script SQL de création des tables dans PostgreSQL. Cette étape est importante parce qu'elle force à poser clairement la structure avant d'écrire la moindre ligne de code Java.

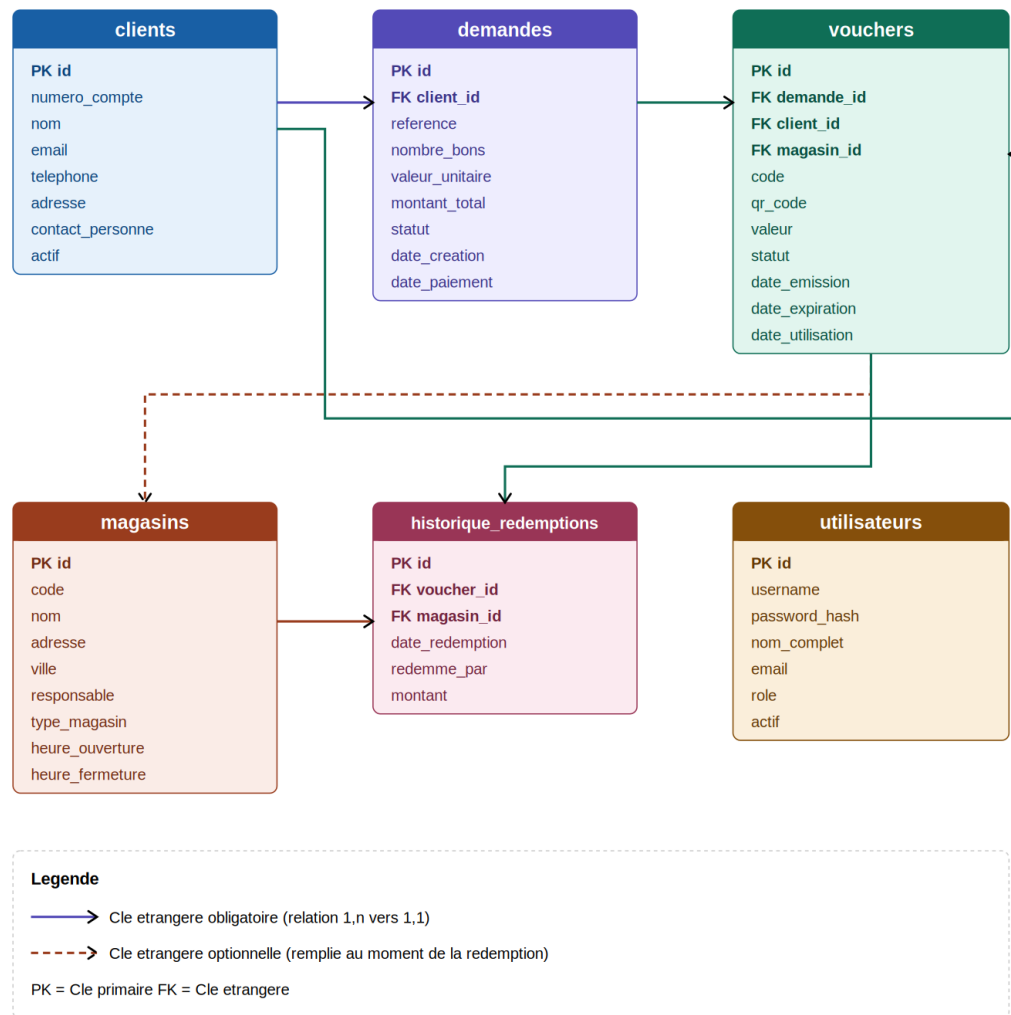


Figure 4 - Schéma relationnel de la base *diary_vms*

Explication du Modèle Conceptuel de Données

1. Les entités principales

Le MCD du VMS tourne autour de six entités principales qui reflètent les différentes dimensions du projet.

- clients : regroupe les entreprises clientes qui passent commande. Chaque client a un numéro de compte unique, un nom, une adresse email, un téléphone, une adresse postale et un contact référent.
- magasins : liste les points de vente partenaires habilités à accepter les vouchers. Un magasin est identifié par son code, avec son nom, sa ville, son adresse, son responsable, son type et ses heures d'ouverture et de fermeture.
- demandes : enregistre chaque commande passée par un client. Une demande contient une référence unique, le nombre de bons souhaités, la valeur unitaire, le montant total et traverse plusieurs statuts successifs : EN_ATTENTE_PAIEMENT, PAYE, APPROUVE, GENERE, ANNULE.
- vouchers : représente chaque bon cadeau individuel. Chaque voucher a son code, son qr_code, sa valeur, sa date d'émission, sa date d'expiration et son statut (GENERE, EMIS, UTILISE, EXPIRE, ANNULE). Il est rattaché à la demande qui l'a généré, au client bénéficiaire, et optionnellement au magasin qui l'a utilisé (via magasin_id et date_utilisation, remplis au moment de la rédemption).
- historique_redemptions : garde une trace détaillée de chaque rédemption. Pour un voucher donné, on y trouve le magasin où il a été utilisé, la date et l'heure, l'utilisateur qui a validé le scan, le montant et éventuellement des remarques.
- utilisateurs : contient les comptes de l'application avec un rôle (INITIATEUR, APPROBATEUR, ADMINISTRATEUR, SUPER_USER). Le mot de passe est stocké sous forme hachée.

2. Les relations entre les entités

- passer (entre clients et demandes) : un client peut passer plusieurs demandes, mais chaque demande est faite par un seul client.
- générer (entre demandes et vouchers) : une demande génère plusieurs vouchers (un par bon), mais chaque voucher n'appartient qu'à une seule demande.
- appartenir (entre clients et vouchers) : chaque voucher est directement rattaché au client qui en est le bénéficiaire, via le champ client_id.
- rédimer (entre vouchers et magasins) : un voucher peut être utilisé dans un magasin donné. Cette relation est optionnelle et n'est remplie qu'au moment de la rédemption, via les champs magasin_id et date_utilisation de la table vouchers.
- tracer (entre vouchers et historique_redemptions) : chaque utilisation d'un voucher ajoute une ligne dans l'historique_redemptions avec le magasin, la date exacte et l'utilisateur qui a scanné.

3. Conclusion

Ce MCD couvre proprement le cycle complet du bon, depuis sa demande jusqu'à sa rédemption, tout en gardant une traçabilité via la table historique_redemptions qui double l'information portée par la table vouchers. Il a servi à générer le script SQL hébergé sur la base diary_vms chez AlwaysData.

b) Description des écrans

- L'écran de CONNEXION

COMPOSITION	FONCTIONNEMENT
Formulaire simple avec un champ identifiant, un champ mot de passe et un bouton de connexion. Logo du projet en haut.	Vérifie les identifiants contre la table utilisateurs. Si la connexion est valide, ouvre le tableau de bord avec les droits correspondant au rôle. Sinon, affiche un message d'erreur.

- L'écran TABLEAU DE BORD

COMPOSITION	FONCTIONNEMENT
Statistiques globales en haut (nombre de demandes, de vouchers émis, de rédemptions, montants), menu latéral donnant accès à chaque module, bouton d'export Excel.	Point d'entrée principal après la connexion. Affiche les chiffres clés calculés à la volée depuis la base et permet de naviguer vers les différents modules.

- L'écran CLIENTS

COMPOSITION	FONCTIONNEMENT
Tableau listant tous les clients avec leurs informations principales, boutons d'ajout, modification et suppression.	Permet de gérer le fichier des clients entreprise. Les modifications sont sauvegardées immédiatement dans PostgreSQL via le ClientDAO.

- L'écran DEMANDES

COMPOSITION	FONCTIONNEMENT
Tableau des demandes avec leur référence, client, nombre de bons, valeur unitaire, statut. Boutons pour créer une nouvelle demande, valider le paiement, approuver, générer les vouchers.	Cœur du flux métier. Chaque ligne traverse les statuts successifs au fur et à mesure des actions. Les boutons disponibles dépendent du statut courant de la demande sélectionnée.

- L'écran VOUCHERS

COMPOSITION	FONCTIONNEMENT
Tableau des vouchers avec code, demande liée, valeur, date d'expiration, statut. Boutons pour générer le PDF individuel, le PDF Groupe d'une demande, et pour envoyer par email.	Permet de consulter l'ensemble des bons émis et d'agir dessus individuellement ou par lot. Le PDF Groupe regroupe tous les bons d'une même demande dans un seul document.

- L'écran RÉDEMPTION

COMPOSITION	FONCTIONNEMENT
Champ pour saisir ou scanner un code voucher, informations détaillées du voucher trouvé, bouton de validation de la rédemption. Le contenu est placé dans un ScrollPane pour bien défiler sur les petits écrans.	Permet au superviseur magasin de valider un bon. Le système interroge la base pour vérifier la validité (non expiré, non déjà utilisé), puis enregistre la rédemption avec le magasin, la date et l'utilisateur.

- L'écran MAGASINS

COMPOSITION	FONCTIONNEMENT
Tableau listant les magasins partenaires avec leur code, nom, ville, responsable.	Permet de gérer les points de vente habilités à accepter les vouchers. Utilisé par l'écran de rédemption pour identifier le magasin où a lieu le scan.

5 – LE DÉVELOPPEMENT

a) Les langages et technologies

- Java 21 / JavaFX 21

Java 21 a été choisi comme langage principal et JavaFX 21 pour la partie interface. JavaFX permet de construire des applications bureau modernes avec une séparation claire entre les fichiers FXML qui décrivent la structure des écrans, les fichiers CSS pour le style, et les classes Java controllers qui gèrent le comportement. C'est une approche assez proche de ce qu'on retrouve en développement web, ce qui rend le code plus facile à organiser.

- PostgreSQL

PostgreSQL a été choisi comme base de données pour son côté robuste, sa gestion fine des contraintes et son support solide des fonctions stockées et des triggers. La base diary_vms est hébergée chez AlwaysData, ce qui permet d'y accéder depuis n'importe quelle machine sans devoir installer un serveur local. L'outil phpPgAdmin fourni par AlwaysData sert à administrer la base au quotidien.

- Maven

Maven est utilisé pour la gestion du projet et des dépendances. Le pom.xml déclare toutes les bibliothèques externes dont dépend le projet (JavaFX, driver PostgreSQL, ZXing, PDFBox, javax.mail, Apache POI, Log4j2) ainsi que les plugins nécessaires à la construction du livrable, notamment maven-shade-plugin pour empaqueter un jar autonome et javafx-maven-plugin pour lancer l'application en développement.

- Bibliothèques externes principales

ZXing est utilisé pour la génération et le décodage des QR codes. PDFBox permet de construire les PDF des vouchers avec leur mise en page personnalisée. La bibliothèque javax.mail gère l'envoi SMTP des emails aux clients, avec un serveur Gmail configuré via un mot de passe d'application. Apache POI est utilisé pour l'export Excel des rapports, avec la construction d'un classeur à plusieurs onglets alimentés par les vues PostgreSQL dédiées. Log4j2 sert à la journalisation des événements de l'application.

- SQL (PostgreSQL)

Le SQL est utilisé directement dans les DAO via JDBC, avec des requêtes préparées pour éviter toute injection. Trois vues ont été créées pour alimenter le reporting Excel : rapport_toutes_demandes, rapport_attente_paiement et rapport_vouchers. Des fonctions stockées sont aussi présentes pour la génération des vouchers et le marquage des bons expirés.

b) Architecture et organisation du code

Plutôt que d'utiliser un framework lourd, le projet suit une architecture classique MVC avec une couche DAO pour l'accès aux données. Concrètement, le code est organisé en plusieurs packages sous `com.vms` :

- `com.vms` : classes racine `Main` et `Launcher`. `Main` étend `Application` et est utilisée par `javafx-maven-plugin` pour le développement. `Launcher` est une classe wrapper qui appelle `Main.main()` et sert de point d'entrée pour `jpackage` (qui refuse les classes étendant `Application`).
- `com.vms.controller` : tous les controllers JavaFX associés aux vues FXML (`LoginController`, `DashboardController`, `ClientsController`, `DemandesController`, `VouchersController`, `RedemptionController`, `MagasinsController`).
- `com.vms.dao` : toute la couche d'accès aux données, avec un DAO par entité (`ClientDAO`, `DemandeDAO`, `MagasinDAO`, `UserDAO`, `VoucherDAO`). Chaque DAO contient ses propres constantes SQL (par exemple `SELECT_BASE` dans `VoucherDAO`) et des méthodes helper du type `extractVoucherFromResultSet` pour éviter la duplication de code entre les méthodes.
- `com.vms.model` : les classes métier qui représentent les entités (`Client`, `Demande`, `Magasin`, `User`, `Voucher`). Elles portent les propriétés JavaFX pour la liaison avec les tableaux.
- `com.vms.util` : les utilitaires (`QRCodeGenerator` avec `ZXing`, `VoucherPDFGenerator` avec `PDFBox`, `EmailSender` avec `javax.mail`, `ExcelExporter` avec `Apache POI`).
- `com.vms.database` : la classe `DatabaseConnection` qui centralise la connexion PostgreSQL avec un mécanisme de réutilisation basé sur `isValid(2)` pour éviter de rouvrir une connexion à chaque action.

Les ressources (fichiers FXML, CSS, images) sont placées dans `src/main/resources/com/vms`. La séparation entre le code Java et les ressources FXML permet de modifier l'apparence des écrans sans toucher au code, ce qui facilite grandement les ajustements d'interface.

c) Les outils utilisés

- IntelliJ IDEA

IntelliJ IDEA a été l'IDE principal pour tout le développement Java. Son intégration native avec Maven, son support de JavaFX avec Scene Builder, son débogueur puissant et ses outils de refactoring en font l'outil idéal pour ce type de projet.

- phpPgAdmin

phpPgAdmin est l'outil fourni par AlwaysData pour administrer la base PostgreSQL. Il a servi à exécuter le script de création des tables, à inspecter les données de test, à créer les vues du reporting et à faire des requêtes SQL manuelles pendant le développement.

- AlwaysData

AlwaysData est l'hébergeur choisi pour la base PostgreSQL. Il fournit un serveur dans le cloud accessible depuis n'importe où, avec phpPgAdmin en interface web, ce qui permet de se passer d'une installation locale et de partager la même base entre plusieurs postes.

- Maven

Maven gère les dépendances et la construction du projet. La commande `mvn clean package` produit le jar empaqueté avec toutes ses dépendances grâce au plugin `maven-shade`.

- jpackage

`jpackage`, inclus dans le JDK à partir de Java 14, a été utilisé pour produire l'application native Windows à partir du jar empaqueté. L'option `--type app-image` a été retenue car elle fonctionne sans nécessiter WiX (obligatoire pour `--type msi`). `jpackage` est appelé via son chemin complet car il n'était pas accessible directement dans le PATH de la machine.

- Git et GitHub

Git et GitHub ont servi pour le versionnement du code et la sauvegarde régulière des évolutions. À noter qu'un point d'attention reste à traiter concernant les identifiants de connexion à la base qui sont encore en dur dans le code : un fichier `db.properties` ajouté au `.gitignore` serait la bonne pratique à mettre en place.

d) Explication des fonctionnalités

➤Côté siège (gestion)

o ÉCRAN DE CONNEXION

L'accès à l'application commence par un écran de connexion. L'utilisateur saisit son identifiant et son mot de passe, qui sont vérifiés contre la table utilisateurs via une requête préparée. Si les identifiants sont corrects, la session est ouverte et le tableau de bord s'affiche avec les droits correspondant au rôle (ADMIN, COMPTABLE, APPROBATEUR, MAGASIN, UTILISATEUR).

o TABLEAU DE BORD

Le tableau de bord sert de point d'entrée après la connexion. Il affiche en haut les statistiques globales calculées à la volée (nombre de demandes en cours, vouchers émis, rédemptions effectuées, montants totaux), un menu latéral donnant accès à chaque module, et un bouton d'export Excel qui permet de générer un rapport complet en un clic.

o GESTION DES CLIENTS ET DES MAGASINS

Ces deux modules fonctionnent de la même manière : un tableau liste toutes les lignes, avec des boutons pour ajouter, modifier ou supprimer une entrée. Chaque modification est immédiatement répercutée

dans PostgreSQL via le DAO correspondant. Pour les clients, un numéro de compte unique est généré et les informations de contact sont enregistrées. Pour les magasins, le code unique permet ensuite d'identifier le point de vente lors des rédemptions.

o GESTION DES DEMANDES

L'écran des demandes est le cœur du flux métier. Un utilisateur crée une nouvelle demande en choisissant le client, le nombre de bons et la valeur unitaire. Une référence unique de type VR0048-200 est générée automatiquement. La demande passe ensuite par plusieurs statuts successifs : EN_ATTENTE_PAIEMENT, PAYE, APPROUVE, puis GENERE quand les vouchers sont émis. Les boutons d'action disponibles dépendent du statut courant, ce qui évite qu'un comptable puisse approuver ou qu'un approbateur génère des bons sans paiement.

o GÉNÉRATION DES VOUCHERS

Quand une demande est approuvée et qu'on lance la génération, le système crée automatiquement un voucher par bon demandé. Chaque voucher reçoit un code unique, un QR code généré par ZXing, une valeur fixe, une date d'émission et une date d'expiration. La bibliothèque PDFBox est ensuite utilisée pour produire un PDF individuel par voucher. Une option PDF Groupe, implémentée via la méthode `getVouchersByDemandId` dans `VoucherDAO`, permet d'obtenir un PDF unique regroupant tous les bons d'une même demande, ce qui est pratique pour l'envoi au client.

o ENVOI EMAIL AUTOMATIQUE

Une fois les PDF générés, la classe `EmailSender` se charge de les envoyer automatiquement au client par email. L'envoi passe par un serveur SMTP Gmail configuré avec un mot de passe d'application spécifique. La mise en place a demandé de résoudre un conflit de dépendances entre Jakarta Mail et javax Mail, résolu en excluant complètement jakarta.mail-api et en standardisant sur com.sun.mail:javax.mail:1.6.2 avec javax.activation:activation:1.1.1. Le plugin shade a aussi été configuré avec `ServicesResourceTransformer` pour que les fichiers de services soient correctement fusionnés.

➤Côté magasin (rédemption)

o ÉCRAN DE RÉDEMPTION

L'écran de rédemption est l'interface utilisée en caisse par le superviseur magasin. Il scanne (ou saisit) le code du voucher présenté par le client. Le système interroge immédiatement la base PostgreSQL pour récupérer le voucher correspondant et vérifier trois choses : que le voucher existe, qu'il n'est pas expiré, et qu'il n'a pas déjà été utilisé. Si tout est bon, le superviseur valide la rédemption, ce qui met à jour le statut du voucher à REDEMME et enregistre le magasin, la date et l'utilisateur. Le contenu central de l'écran est enveloppé dans un `ScrollPane` pour bien défiler sur des écrans de taille réduite.

➤Reporting

o EXPORT EXCEL

Depuis le tableau de bord, un bouton permet de générer un rapport Excel complet. Côté base, trois vues PostgreSQL ont été créées pour alimenter le rapport : `rapport_toutes_demands`, `rapport_attente_paiement` et `rapport_vouchers`. Côté Java, la classe `ExcelExporter` utilise Apache POI pour construire un fichier Excel à trois onglets, un par vue, avec les en-têtes en gras et les données formatées. Ce rapport permet à la direction d'avoir une vision globale à tout moment sans avoir besoin d'une connexion directe à la base.

➤Packaging

o APPLICATION NATIVE WINDOWS

Pour faciliter le déploiement, l'application est empaquetée en native Windows via `jpackage`. Une classe `Launcher` distincte appelle la méthode `main` de `Main`, ce qui contourne la limite de `jpackage` qui refuse les classes étendant `Application`. Deux configurations distinctes sont maintenues : `maven-shade-plugin` pointe vers `com.vms.Launcher` pour produire le jar empaqueté utilisé par `jpackage`, tandis que `javafx-maven-plugin` garde `com.vms.Main` pour le lancement en développement. Les deux ne doivent pas être unifiés car ils servent des usages différents.

6 – CONCLUSION

La réalisation du projet VMS a été une vraie expérience de développement d'application bureau complète, qui m'a fait passer par toutes les étapes d'un projet réel : analyse du contexte métier, conception de la base de données, développement de l'interface JavaFX, intégration des bibliothèques externes (QR codes, PDF, emails, Excel), tests, empaquetage et documentation.

Ce qui rend ce projet particulier à mes yeux, c'est le passage de la première à la deuxième année. En première année on était 3 à 4 sur le projet, l'application fonctionnait globalement mais avec du code qui n'était pas toujours très propre : duplication dans les DAO, connexion à la base qui se rouvrait à chaque clic, conflits de bibliothèques SMTP qui empêchaient l'envoi des emails, fonctionnalités de reporting qui manquaient. En deuxième année, j'ai repris le projet seul et j'ai pu remettre les choses à plat : refactorisation complète des DAO avec des méthodes helper et des constantes SQL réutilisables, correction du problème de reconnexion en sortant la connexion du try-with-resources, résolution du conflit Jakarta/javax Mail, ajout de la fonctionnalité PDF Groupe, mise en place du reporting Excel via trois vues PostgreSQL dédiées, et empaquetage de l'application en native Windows via jpackage.

Au-delà du résultat, c'est surtout la démarche qui a été formatrice : apprendre à relire du code avec du recul, à repérer ce qui ne va pas, à réécrire proprement sans tout casser, à réfléchir en termes de maintenabilité et de réutilisation. Le passage d'un code qui marche à un code qui est bien fait demande beaucoup plus de temps qu'on ne l'imagine, mais c'est là que se joue la vraie différence entre un projet d'école et un projet qu'on pourrait livrer à un vrai client.

Pour résumer, ce projet m'a permis de consolider mes bases en Java et JavaFX, d'apprendre à intégrer plusieurs bibliothèques externes dans un même projet sans qu'elles se marchent dessus, de manipuler PostgreSQL au-delà des simples requêtes (vues, fonctions stockées, triggers), et de comprendre concrètement ce que veulent dire les principes de factorisation et de DRY dans un vrai projet. Il reste encore des pistes d'amélioration, notamment la sécurisation des identifiants de connexion qui sont actuellement en dur dans le code, et la mise en place d'un pool de connexions type HikariCP pour mieux gérer l'accès concurrent à la base.

7 – ANNEXE

a) Les références utilisées

- « Contexte de la Réalisation Professionnelle No. 2 - VMS », document fourni par MCCI Business School (Cybercité, Ebène - Île Maurice), rédigé par le chargé de cours Yusuf Serally.
- Documentation officielle JavaFX : <https://openjfx.io/>
- Documentation officielle PostgreSQL : <https://www.postgresql.org/docs/>
- Documentation ZXing (QR codes) : <https://github.com/zxing/zxing>
- Documentation Apache PDFBox : <https://pdfbox.apache.org/>
- Documentation JavaMail API : <https://javaee.github.io/javamail/>
- Documentation Apache POI : <https://poi.apache.org/>
- Documentation Log4j2 : <https://logging.apache.org/log4j/2.x/>
- Documentation Maven : <https://maven.apache.org/guides/>
- Documentation jpackage : <https://docs.oracle.com/en/java/javase/21/jpackage/>
- Documentation AlwaysData : <https://help.alwaysdata.com/fr/>
- Cours et supports du BTS SIO option SLAM — notions de développement Java, bases de données et conception.

b) Apprentissage

Les compétences acquises grâce à ce projet :

- Développement bureau avec JavaFX :

- Séparation claire entre vues FXML, styles CSS et controllers Java, liaison de données avec les propriétés observables, gestion des événements utilisateur, navigation entre écrans.

- Accès aux bases de données avec JDBC :

- Écriture de DAO propres avec requêtes préparées, gestion des connexions via un singleton avec vérification isValid, mise en place de méthodes helper pour éviter la duplication dans l'extraction des ResultSet.

- Conception et administration PostgreSQL :

- Élaboration du MCD, écriture du script SQL de création, gestion via phpPgAdmin, création de vues pour le reporting, utilisation de fonctions stockées pour la logique métier lourde.

- Intégration de bibliothèques externes :

- ZXing pour les QR codes, PDFBox pour les PDF, javax.mail pour l'envoi SMTP, Apache POI pour Excel, Log4j2 pour les logs. Apprentissage de la résolution de conflits de dépendances entre bibliothèques qui partagent le même espace de nommage.

- Factorisation et organisation du code :

- Mise en place de méthodes helper dans les DAO, centralisation des constantes SQL, séparation claire entre couches modèle, accès données et contrôleur.

- Gestion de projet Maven :

- Écriture d'un pom.xml complet avec dépendances et plugins, configuration du maven-shade-plugin pour produire un jar autonome, utilisation du javafx-maven-plugin pour le mode développement, compréhension de la différence entre les deux points d'entrée Main et Launcher.

- Empaquetage et déploiement :

- Utilisation de jpackage avec --type app-image pour produire une application native Windows autonome, gestion de l'hébergement de la base sur AlwaysData, tests de la connexion cloud depuis différents postes.

- Documentation :

- Rédaction de ce rapport de projet, documentation technique des choix d'architecture, commentaires clairs dans le code aux endroits où la logique mérite d'être explicitée.